

```
int square(int x)
{
    return x*x;
}
```

To build the above code, i.e., compile individual translation units and link generated object files, you can use the following sequence of commands:

```
gcc test.c -c -o test.o
gcc sum.c -c -o sum.o
gcc sqr.c -c -o sqr.o
gcc test.o sum.o sqr.o -o all.out
#all.exe in case of windows
```

Static vs dynamic linking

Applications may be linked statically or dynamically. In static linking, all necessary code is kept part of executable by the linker, which enables better performance, eliminating runtime overhead. But this poses the problem of a larger footprint of executables. In dynamic linking, library functions are excluded in the executable and loaded on demand, which allows an optimal footprint, but poses the problem of runtime overhead. Dynamic libraries come with the added advantage of sharing among applications and versioning support. A library is a collection of object files. Let's create libraries for the frequently used functions like `sum` and `square` in the above code.

```
ar rc libsample.a sum.o sqr.o
#static libraries come with extension .a, lib prefix is a
convention
gcc -l. -lsample -o p.out
#libsample.a is linked statically and other std libraries
like libc.so, libm.so are linked dynamically

gcc -l. -lsample -o s.out -static
# all libraries are linked statically, glibc-devel-static
package is required in Linux for static linking of std libs
like libc.a, libm.a

# Assume .exe extension instead of .out in windows,
# MingW suite doesn't have much dynamic libs, most of the
linking happens statically here.

gcc -shared sum.o sqr.o -o libsample.so
#On Linux shared object(.so) files support dynamic linking
gcc -shared sum.o sqr.o -o libsample.dll
#On Windows dynamic link libraries(dll) format is used
gcc -l. test.o -lsample -o d.out
#Link with libsample.* dynamically
```

While linking, the `-L` option is used by `gcc` to specify the custom path of our own libraries, and `-l` is used to specify

the custom libraries, assuming the `lib` prefix and the `.a` or `.so` extension. In the above commands, a dot (`.`) is specified with `-L`, as the necessary libraries are in the current directory; so replace the dot (`.`) with the concerned path, as applicable. For example, `-lc` stands for `libc.a` or `libc.so`, `-lpthread` stands for `libpthread.*`. We use `-lsample` to specify `libsample.a` or `libsample.so`.

When both `.a` and `.so` are available in a specified directory, `gcc` opts for `.so` files by default. In Linux, the `-static` option is used to enforce static linking.

Compare the footprint of `s.out`, `p.out` and `d.out` using the `size` command or OS-specific commands like `ls`, `du` or `dir`. You will observe that `s.out` is heavy with all the library code and `d.out` is very light with minimal code, while `p.out` comes inbetween as only our library is statically linked. The strip utility can be applied on executables to reduce the footprint, which removes symbols from underlying object files. Please note that the strip can't be applied on individual object files or libraries before linking. It is meaningful for executables only, especially statically linked code for constrained environments.

`strip s.out#compare the size of s.out before and after strip`



Note: The following section is Linux specific.

```
First, type:
gcc -shared -Wl,-soname,libsample.so -o ~/dlibs/libsample.
so.1.0.1 sum.o sqr.o
ldconfig -n ~/dlibs
```

To run the dynamically linked executable, we need to specify the custom path of the libraries:

```
LD_LIBRARY_PATH=~:/dlibs ./d.out
```

Alternatively, we need to update the `LD_LIBRARY_PATH` environment variable appended with `~/dlibs`. It would be preferable if you could add an entry of the custom directory in `/etc/ld.so.conf` and run `ldconfig` once, to update the cache.

To check the shared library dependencies of any executable, we can use `ldd`, as follows:

```
ldd p.out d.out
#our library is listed in d.out but not in p.out
ldd s.out
#says no dependencies
```

Please refer to tldp.org/HOWTO/Program-Library-HOWTO/shared-libraries.html for more details on shared object files.

A simple Linux utility `file` mentions the type of each file, particularly for binaries, providing helpful details like target architecture, whether it is statically or dynamically linked and if it is stripped or not.